Universidad Nacional de Ingeniería

Área de Conocimiento de Tecnología de la Información y Comunicación

Carrera: Ingeniería en computación

Propuesta de Proyecto Final: Sistema de Gestión Académica Integral (SGAI) para el Colegio Antonio Soldano del municipio de Matiguás, Departamento de Matagalpa.

Proyecto de fin de curso
Asignatura: Algoritmización y estructura de datos

Elaborado por:

Nelson Antonio Blandón
Carné: 2017-1392U

Ingrid Judith Orozco Martínez
Carné: 2025-0641U

Danny Benito Rojas González
Carné: 2025-0070U

Grupo:

2V1-COM-S

Presentado a:

Ing. Yahaira Andrea Altamirano Briceño

12 de octubre de 2026
Managua, Nicaragua

Universidad
Nacional de
Ingeniería

Universidad Nacional de Ingeniería

1. INTRODUCCIÓN

El presente documento constituye la propuesta formal y técnica del Proyecto Final para la asignatura de Algoritmización y Estructura de Datos (AED) de la carrera de Ingeniería en Computación en la Universidad Nacional de Ingeniería (UNI). El proyecto aborda la problemática del Colegio Antonio Soldano mediante la concepción, diseño e implementación de un Sistema de Gestión Académica.

El Sistema de Gestión Académica Integral (SGAI) es una aplicación de escritorio que permitirá la administración completa del Colegio Antonio Soldano. El sistema permitirá a los usuarios (Administradores, Docentes, Estudiantes) gestionar roles, registrar información de personal docente y alumnado, asignar cursos e ingresar calificaciones.

A nivel lógico, el sistema operará bajo un modelo transaccional en memoria: extraerá los datos de la base de datos y los instanciará en listas dinámicas, permitiendo realizar manipulaciones algorítmicas (inserción, eliminación y reportes ordenados) directamente sobre objetos en Java. Finalmente, persistirá estos cambios hacia una base de datos relacional de manera asíncrona.

En este documento, se abordan los antecedentes del centro de estudios, la justificación de la automatización, los objetivos del sistema, los requerimientos del sistema, el diseño estructural mediante Diagramas de Clases UML y Diagramas Entidad-Relación (E-R).

2. ANTECEDENTES

A lo largo de las últimas décadas, los **Sistemas de Información** han transformado radicalmente la gestión administrativa y operativa dentro de los entornos institucionales, desplazando los antiguos métodos de registro basados en soporte físico y auditorías manuales. En el ámbito de la **Ingeniería en Computación**, la evolución desde el almacenamiento en archivos tradicionales hacia sistemas automatizados persistentes representa un estándar fundamental para garantizar la integridad, centralización y disponibilidad de los datos en tiempo real.

Universidad Nacional de Ingeniería

Para el desarrollo de la presente investigación y propuesta tecnológica orientada al **Colegio Antonio Soldano**, nuestro equipo —integrado por **Nelson Blandón, Judith Orozco y Danny Rojas**— ha tomado como base técnica y metodológica la experiencia acumulada en proyectos de software desarrollados en semestres anteriores dentro de la **Universidad Nacional de Ingeniería (UNI), así como trabajos de investigación en proyectos de conclusión de curso, propuestos por estudiantes en universidades internacionales**. En este sentido, nuestro trabajo de proyecto e investigación se fundamenta en la identificación y análisis de los siguientes antecedentes nacionales e internacionales:

Antecedentes nacionales:

1. **Sistema de Registro Académico para estudiantes de 2V1-COM - UNI:** Prototipo de backend desarrollado individualmente por **Nelson Blandón** mediante el paradigma de **Programación Orientada a Objetos (POO)** en Java, enfocado en el procesamiento de matrículas y el registro persistente de calificaciones a través de conectores relacionales de bases de datos.

2. **Sistema de Gestión Académico Colegio Antonio Soldano** Prototipo orientado al control de perfiles docentes, asignación de cursos y el diseño de módulos de seguridad para la de autenticación de usuarios.

La evaluación técnica y diagnóstica de estas propuestas previas evidenció limitaciones críticas asociadas al acoplamiento estricto entre la interfaz gráfica y el motor lógico, la falta de normalización en esquemas relacionales (como la duplicación de estructuras por niveles o grados) y la vulnerabilidad ante inyecciones de código mediante la concatenación directa de cadenas de texto.

Antecedentes internacionales:

1. **Enfoque en Gestión e Interfaces:** En la investigación titulada *"Análisis, diseño, desarrollo e implementación del sistema de gestión académica integral*

(SISGACI)" de la **Universidad Nacional Mayor de San Marcos (Perú) de Gutara B. Et, al**, se abordó la automatización de procesos escolares mediante metodologías de desarrollo de software estructuradas. Este estudio técnico demostró la importancia de erradicar los registros aislados y centralizar los flujos de matrícula mediante capas lógicas independientes, aportando validez al diseño modular aplicado para el Colegio Antonio Soldano.

2. **Enfoque en Estructuras de Datos y Optimización:** En el estudio especializado sobre *"Optimización de sistemas de gestión académica. Una propuesta de gestión, medición y procesamiento de datos en un entorno virtual de aprendizaje para la toma de decisiones en instituciones educativas"* de la **Universidad de Estudios de Administración de Negocios, de Bogotá, Colombia**, de Ortegón, G., se analizó el impacto del tiempo de respuesta del servidor y la manipulación de datos en memoria RAM durante consultas concurrentes. Esta investigación valida metodológicamente nuestra propuesta de integrar listas enlazadas dinámicas en memoria volátil para acelerar los algoritmos de ordenamiento y filtrado de estudiantes, sin sobrecargar las transacciones persistentes en el motor de base de datos relacional.

A partir del análisis de las investigaciones recopiladas, se concluye metodológicamente que el compendio de estos antecedentes proporciona las bases lógicas y los requerimientos esenciales para la delimitación del problema objeto de estudio. De cara al diseño técnico, estos referentes fundamentan de manera analítica la viabilidad de desarrollar e implementar el **Sistema de Gestión Académico Integral** orientado al **Colegio Antonio Soldano**.

Dicha propuesta se concibe con miras a proyectar una solución alternativa de software que optimice de forma sistemática la gestión de los registros, el flujo de matrículas y los procesos académicos correspondientes al claustro administrativo, cuerpo docente, estudiantes y personal directivo de la institución. Por consiguiente, la articulación de estos antecedentes nacionales e internacionales contribuye a justificar la transición hacia un modelo informático autónomo que combina la persistencia relacional con el

rendimiento de estructuras de datos dinámicas en memoria RAM para asegurar la consistencia del entorno educativo evaluado.

3. JUSTIFICACIÓN

La automatización de procesos mediante un sistema de información moderno representa una ventaja competitiva decisiva para el Centro de Estudios. Se identificó la necesidad urgente de automatizar y optimizar la gestión académica por las siguientes razones clave:

Eficiencia Algorítmica en Memoria: La migración hacia estructuras de datos dinámicas (Listas Enlazadas Simples y Dobles) permite realizar inserciones, ordenamientos (QuickSort/Burbuja) y búsquedas sobre la memoria RAM sin saturar el motor de base de datos con consultas recurrentes ORDER BY o GROUP BY.

Normalización e Integridad de Datos: El rediseño de la base de datos elimina la duplicidad de registros (unificando la gestión de cursos) y garantiza la atomicidad de los datos de docentes y estudiantes bajo la Primera Forma Normal (1FN).

Seguridad y Robustez: Reemplaza la ejecución de consultas concatenadas vulnerables a Inyección SQL (SQLi) por el uso seguro de PreparedStatement mediante el patrón Data Access Object (DAO).

Escalabilidad Operativa: La arquitectura desacoplada permite procesar volúmenes crecientes de estudiantes y calificaciones mediante sincronización y persistencia asíncrona a la base de datos relacional.

Universidad Nacional de Ingeniería

4. OBJETIVOS

Objetivo General

Desarrollar un Sistema de Gestión Académica Integral para el Colegio Antonio Soldano que permita la administración de estudiantes, docentes, roles, cursos y reportes aplicando estructuras de datos dinámicas para el procesamiento en memoria, garantizando una arquitectura de capas y asegurando la persistencia asincrónica en una base de datos relacional normalizada.

Objetivos Específicos

- Implementar listas enlazadas simples y compuestas para gestionar los nodos de estudiantes, docentes y el historial de calificaciones dinámico.
- Desarrollar algoritmos de ordenamiento manuales (ej. QuickSort o Burbuja) y algoritmos de búsqueda directamente sobre los punteros de las estructuras en memoria para la generación de reportes.
- Diseñar una base de datos normalizada aplicando la Primera Forma Normal (1FN) y garantizando la integridad referencial.
- Aplicar una arquitectura de software limpia (MVC o DAO) y segura, previniendo vulnerabilidades como la Inyección SQL.

Universidad Nacional de Ingeniería

5. REQUERIMIENTOS DEL SISTEMA

5.1. Funciones Principales

- **Gestión de usuarios y roles:** autenticar usuarios y asignar permisos según los roles Administrador, Docente y Estudiante.
- **Gestión de estudiantes:** registrar, consultar, modificar y eliminar expedientes, con número de carné único.
- **Gestión de docentes, niveles y cursos:** administrar el catálogo académico y asignar cursos al personal docente.
- **Registro de calificaciones:** registrar y actualizar notas asociadas a un estudiante y un curso.
- **Procesamiento de reportes:** generar boletines y clasificaciones de promedios mediante algoritmos de ordenamiento en memoria.

5.2. Usuarios del Sistema

- **Administrador:** acceso a la configuración, gestión de usuarios, asignación de roles y administración de los datos.
- **Docente:** consulta de los cursos asignados, registro y modificación autorizada de calificaciones y generación de actas.
- **Estudiante:** consulta de su historial académico, cursos y boletín de calificaciones.

5.3. Módulos del Sistema

- **Autenticación y seguridad:** controla el acceso mediante credenciales y roles.
- **Administración académica:** gestiona la información de estudiantes, docentes, niveles y cursos mediante DAO.
- **Estructuras dinámicas en memoria:** mantiene listas enlazadas de estudiantes, docentes y calificaciones para operaciones CRUD.
- **Algoritmos y reportes:** ejecuta búsquedas, ordenamientos y filtrados sobre las estructuras.

- **Persistencia:** sincroniza los cambios con MySQL mediante operaciones parametrizadas y transacciones cuando corresponda.

6. DIAGRAMA DE CLASES Y DIAGRAMA ENTIDAD-RELACIÓN

6.1. Diagrama de Clases (UML)

El diseño del sistema se estructura en tres capas principales:

- **Capa de Dominio (POO):**

Clase	Atributos principales	Correspondencia / regla
Persona (abstracta)	- nombres: String - apellidos: String	Clase base para los perfiles personales; no incluye documentoIdentidad porque ese atributo no existe en las tablas actuales.
Estudiante extends Persona	- idEstudiante: int - carnet: String - idUsuario: Integer	idUsuario es opcional y único cuando está informado.
Docente extends Persona	- idDocente: int - especialidad: String - idUsuario: Integer	idUsuario es opcional y único cuando está informado.
Usuario	- idUsuario: int - username: String - password: String (hash) - rol: String / enum	La contraseña debe contener un hash Argon2id o bcrypt generado por la aplicación, nunca texto plano en producción.
NivelAcademico	- idNivel: int - nombreNivel: String	Representa la tabla niveles_academicos.
Curso	- idCurso: int - nombreCurso: String - idNivel: int	Cada curso pertenece a un nivel académico existente.
AsignacionDocenteCurso	- idAsignacion: int - idDocente: int - idCurso: int	Entidad asociativa; la pareja docente-curso es única.
Calificacion	- idCalificacion: int - idEstudiante: int - idCurso: int - valorNota: BigDecimal - fechaRegistro: LocalDate	BigDecimal corresponde a DECIMAL(5,2); la nota se restringe al rango 0–100 según la escala asumida.

- **Capa de Estructuras Dinámicas (AED):**
  - `NodoEstudiante:` Atributos `-estudiante: Estudiante`, `-siguiente: NodoEstudiante`, `-notas: ListaEnlazadaCalificaciones`.

- `ListaEnlazadaEstudiantes`: Métodos `+insertarAlFinal()`, `+ordenarQuickSort()`, `+buscarPorCarnet()`.
- `NodoCalificacion` y `ListaEnlazadaCalificaciones`: administran las calificaciones asociadas a un estudiante. Cada calificación mantiene además sus identificadores de estudiante y curso para la persistencia.

Las estructuras en memoria son una representación operativa de los datos; no sustituyen las claves ni las restricciones de la base de datos.

- **Capa de Persistencia:**
  - **ConexionDB:** administra la conexión JDBC a MySQL.
  - **DAO** (por ejemplo, UsuarioDAO, EstudianteDAO, DocenteDAO, CursoDAO y CalificacionDAO): ejecuta operaciones CRUD mediante PreparedStatement y transacciones cuando sean necesarias.
  - **SistemaRegistro:** coordina las operaciones de aplicación y la sincronización entre las estructuras dinámicas y la base de datos.

6.2. Modelo Entidad Relación

El modelo relacional consta de las siguientes entidades y restricciones:

Entidad	Atributos y restricciones principales
USUARIOS	PK id_usuario; username UNIQUE; password; rol
ESTUDIANTES	PK id_estudiante; carnet UNIQUE; nombres; apellidos; FK id_usuario UNIQUE, NULL permitido
DOCENTES	PK id_docente; nombres; apellidos; especialidad; FK id_usuario UNIQUE, NULL permitido
NIVELES_ACADEMICOS	PK id_nivel; nombre_nivel UNIQUE
CURSOS	PK id_curso; nombre_curso; FK id_nivel; UNIQUE (nombre_curso, id_nivel)

ASIGNACION_DOCENTE_CURSO	PK id_asignacion; FK id_docente; FK id_curso; UNIQUE (id_docente, id_curso)
CALIFICACIONES	PK id_calificacion; FK id_estudiante; FK id_curso; nota DECIMAL(5,2), CHECK 0–100; fecha_registro DATE

7. SCRIPT SQL

El siguiente script crea el esquema para una instalación nueva. Las sentencias CREATE TABLE IF NOT EXISTS no modifican tablas preexistentes; si la base de datos anterior ya existe, se debe aplicar una migración controlada antes de ejecutar este script. Los datos de acceso incluidos son exclusivamente de prueba y no deben utilizarse en producción.

```
-- SGAI SOLDANO - MySQL 8.0
CREATE DATABASE IF NOT EXISTS sga_soldano_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE sga_soldano_db;


CREATE TABLE IF NOT EXISTS usuarios (
  id_usuario INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  rol ENUM('ADMINISTRADOR','DOCENTE','ESTUDIANTE') NOT NULL
) ENGINE=InnoDB;


CREATE TABLE IF NOT EXISTS niveles_academicos (
  id_nivel INT AUTO_INCREMENT PRIMARY KEY,
  nombre_nivel VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB;


CREATE TABLE IF NOT EXISTS estudiantes (
  id_estudiante INT AUTO_INCREMENT PRIMARY KEY,
  carnet VARCHAR(20) NOT NULL UNIQUE,
  nombres VARCHAR(100) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  id_usuario INT NULL UNIQUE,
  CONSTRAINT fk_estudiantes_usuarios FOREIGN KEY (id_usuario)
    REFERENCES usuarios(id_usuario) ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB;


CREATE TABLE IF NOT EXISTS docentes (
  id_docente INT AUTO_INCREMENT PRIMARY KEY,
  nombres VARCHAR(100) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  especialidad VARCHAR(100) NOT NULL,
  id_usuario INT NULL UNIQUE,
  CONSTRAINT fk_docentes_usuarios FOREIGN KEY (id_usuario)
    REFERENCES usuarios(id_usuario) ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB;


CREATE TABLE IF NOT EXISTS cursos (
  id_curso INT AUTO_INCREMENT PRIMARY KEY,
```

Universidad Nacional de Ingeniería

```
  nombre_curso VARCHAR(100) NOT NULL,
  id_nivel INT NOT NULL,
  CONSTRAINT uq_curso_nivel UNIQUE (nombre_curso, id_nivel),
  CONSTRAINT fk_cursos_niveles FOREIGN KEY (id_nivel)
    REFERENCES niveles_academicos(id_nivel) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS asignacion_docente_curso (
  id_asignacion INT AUTO_INCREMENT PRIMARY KEY,
  id_docente INT NOT NULL,
  id_curso INT NOT NULL,
  CONSTRAINT uq_docente_curso UNIQUE (id_docente, id_curso),
  CONSTRAINT fk_asignacion_docente FOREIGN KEY (id_docente)
    REFERENCES docentes(id_docente) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_asignacion_curso FOREIGN KEY (id_curso)
    REFERENCES cursos(id_curso) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS calificaciones (
  id_calificacion INT AUTO_INCREMENT PRIMARY KEY,
  id_estudiante INT NOT NULL,
  id_curso INT NOT NULL,
  nota DECIMAL(5,2) NOT NULL,
  fecha_registro DATE NOT NULL,
  CONSTRAINT chk_calificacion_nota CHECK (nota >= 0 AND nota <= 100),
  CONSTRAINT fk_calificaciones_estudiante FOREIGN KEY (id_estudiante)
    REFERENCES estudiantes(id_estudiante) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_calificaciones_curso FOREIGN KEY (id_curso)
    REFERENCES cursos(id_curso) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Niveles de prueba: reemplazar por la nomenclatura institucional real.
INSERT INTO niveles_academicos (nombre_nivel)
VALUES ('Nivel 1'), ('Nivel 2')
ON DUPLICATE KEY UPDATE nombre_nivel = VALUES(nombre_nivel);

-- Usuarios de prueba. En producción, password debe contener un hash Argon2id/bcrypt.
INSERT INTO usuarios (username, password, rol) VALUES
 ('admin','admin123','ADMINISTRADOR'),
 ('doc_mendoza','doc123','DOCENTE'),
 ('est_rojas','est123','ESTUDIANTE'),
 ('est_blandon','est456','ESTUDIANTE'),
 ('est_orozco','est789','ESTUDIANTE')
ON DUPLICATE KEY UPDATE username = VALUES(username);

-- Perfiles: identificadores de usuario obtenidos por username.
INSERT INTO estudiantes (carnet,nombres,apellidos,id_usuario)
SELECT '2025-0070U','Danny Benito','Rojas González',id_usuario
FROM usuarios WHERE username='est_rojas'
AND NOT EXISTS (SELECT 1 FROM estudiantes WHERE carnet='2025-0070U');
INSERT INTO estudiantes (carnet,nombres,apellidos,id_usuario)
SELECT '2017-1392U','Nelson Antonio','Blandón',id_usuario
FROM usuarios WHERE username='est_blandon'
AND NOT EXISTS (SELECT 1 FROM estudiantes WHERE carnet='2017-1392U');
INSERT INTO estudiantes (carnet,nombres,apellidos,id_usuario)
SELECT '2025-0641U','Ingrid Judith','Orozco Martínez',id_usuario
FROM usuarios WHERE username='est_orozco'
AND NOT EXISTS (SELECT 1 FROM estudiantes WHERE carnet='2025-0641U');

-- Se conserva el vínculo de usuario del esquema de origen.
INSERT INTO docentes (nombres,apellidos,especialidad,id_usuario)
SELECT 'Yahaira Andrea','Altamirano Briceño','Algoritmos y Estructuras de Datos',id_usuario
FROM usuarios WHERE username='doc_mendoza'
AND NOT EXISTS (SELECT 1 FROM docentes WHERE nombres='Yahaira Andrea' AND apellidos='Altamirano Briceño');
INSERT INTO docentes (nombres,apellidos,especialidad,id_usuario)
SELECT 'Carlos Alberto','Mendoza Pérez','Bases de Datos',NULL
WHERE NOT EXISTS (SELECT 1 FROM docentes WHERE nombres='Carlos Alberto' AND apellidos='Mendoza Pérez');

INSERT INTO cursos (nombre_curso,id_nivel)
SELECT 'Algoritmización y Estructura de Datos',id_nivel FROM niveles_academicos
```

```
WHERE nombre_nivel='Nivel 2' AND NOT EXISTS (SELECT 1 FROM cursos WHERE nombre_curso='Algoritmización y Estructura de
Datos' AND id_nivel=niveles_academicos.id_nivel);
INSERT INTO cursos (nombre_curso,id_nivel)
SELECT 'Programación Orientada a Objetos',id_nivel FROM niveles_academicos
WHERE nombre_nivel='Nivel 1' AND NOT EXISTS (SELECT 1 FROM cursos WHERE nombre_curso='Programación Orientada a
Objetos' AND id_nivel=niveles_academicos.id_nivel);
INSERT INTO cursos (nombre_curso,id_nivel)
SELECT 'Bases de Datos I',id_nivel FROM niveles_academicos
WHERE nombre_nivel='Nivel 2' AND NOT EXISTS (SELECT 1 FROM cursos WHERE nombre_curso='Bases de Datos I' AND
id_nivel=niveles_academicos.id_nivel);


INSERT INTO asignacion_docente_curso (id_docente,id_curso)
SELECT d.id_docente,c.id_curso FROM docentes d JOIN cursos c ON c.nombre_curso='Algoritmización y Estructura de Datos'
WHERE d.nombres='Yahaira Andrea' AND d.apellidos='Altamirano Briceño'
AND NOT EXISTS (SELECT 1 FROM asignacion_docente_curso a WHERE a.id_docente=d.id_docente AND a.id_curso=c.id_curso);
INSERT INTO asignacion_docente_curso (id_docente,id_curso)
SELECT d.id_docente,c.id_curso FROM docentes d JOIN cursos c ON c.nombre_curso='Bases de Datos I'
WHERE d.nombres='Carlos Alberto' AND d.apellidos='Mendoza Pérez'
AND NOT EXISTS (SELECT 1 FROM asignacion_docente_curso a WHERE a.id_docente=d.id_docente AND a.id_curso=c.id_curso);


-- Calificaciones de prueba. NOT EXISTS evita repetir exactamente estos registros.
INSERT INTO calificaciones (id_estudiante,id_curso,nota,fecha_registro)
SELECT e.id_estudiante,c.id_curso,95.50,'2026-10-01' FROM estudiantes e JOIN cursos c ON
c.nombre_curso='Algoritmización y Estructura de Datos'
WHERE e.carnet='2025-0070U' AND NOT EXISTS (SELECT 1 FROM calificaciones x WHERE x.id_estudiante=e.id_estudiante AND
x.id_curso=c.id_curso AND x.nota=95.50 AND x.fecha_registro='2026-10-01');
INSERT INTO calificaciones (id_estudiante,id_curso,nota,fecha_registro)
SELECT e.id_estudiante,c.id_curso,88.00,'2026-06-15' FROM estudiantes e JOIN cursos c ON c.nombre_curso='Programación
Orientada a Objetos'
WHERE e.carnet='2025-0070U' AND NOT EXISTS (SELECT 1 FROM calificaciones x WHERE x.id_estudiante=e.id_estudiante AND
x.id_curso=c.id_curso AND x.nota=88.00 AND x.fecha_registro='2026-06-15');
INSERT INTO calificaciones (id_estudiante,id_curso,nota,fecha_registro)
SELECT e.id_estudiante,c.id_curso,92.00,'2026-10-01' FROM estudiantes e JOIN cursos c ON
c.nombre_curso='Algoritmización y Estructura de Datos'
WHERE e.carnet='2017-1392U' AND NOT EXISTS (SELECT 1 FROM calificaciones x WHERE x.id_estudiante=e.id_estudiante AND
x.id_curso=c.id_curso AND x.nota=92.00 AND x.fecha_registro='2026-10-01');
INSERT INTO calificaciones (id_estudiante,id_curso,nota,fecha_registro)
SELECT e.id_estudiante,c.id_curso,98.00,'2026-10-01' FROM estudiantes e JOIN cursos c ON
c.nombre_curso='Algoritmización y Estructura de Datos'
WHERE e.carnet='2025-0641U' AND NOT EXISTS (SELECT 1 FROM calificaciones x WHERE x.id_estudiante=e.id_estudiante AND
x.id_curso=c.id_curso AND x.nota=98.00 AND x.fecha_registro='2026-10-01');
INSERT INTO calificaciones (id_estudiante,id_curso,nota,fecha_registro)
SELECT e.id_estudiante,c.id_curso,90.50,'2026-10-02' FROM estudiantes e JOIN cursos c ON c.nombre_curso='Bases de
Datos I'
WHERE e.carnet='2025-0641U' AND NOT EXISTS (SELECT 1 FROM calificaciones x WHERE x.id_estudiante=e.id_estudiante AND
x.id_curso=c.id_curso AND x.nota=90.50 AND x.fecha_registro='2026-10-02');
```

8. CONSULTAS SQL

a. SELECT básico: listar estudiantes

```
SELECT * FROM estudiantes;
```

b. Filtrar calificaciones de 90 o más

```
SELECT * FROM calificaciones WHERE nota >= 90.00;
```

c. Estudiantes con sus cuentas de usuario

```
SELECT e.carnet, e.nombres, e.apellidos, u.username, u.rol
FROM estudiantes e
LEFT JOIN usuarios u ON e.id_usuario = u.id_usuario;
```

d. Boletín de calificaciones

```
SELECT e.carnet, CONCAT(e.nombres, ' ', e.apellidos) AS estudiante,
       c.nombre_curso, cal.nota, cal.fecha_registro
FROM calificaciones cal
```

```
JOIN estudiantes e ON cal.id_estudiante = e.id_estudiante
JOIN cursos c ON cal.id_curso = c.id_curso;
```

e. Promedio por estudiante, de mayor a menor

```
SELECT e.carnet, e.nombres, e.apellidos, AVG(cal.nota) AS promedio
FROM estudiantes e JOIN calificaciones cal ON e.id_estudiante = cal.id_estudiante
GROUP BY e.id_estudiante, e.carnet, e.nombres, e.apellidos
ORDER BY promedio DESC;
```

f. Estudiantes con promedio de al menos 90

```
SELECT e.carnet, CONCAT(e.nombres, ' ', e.apellidos) AS estudiante, AVG(cal.nota) AS promedio
FROM estudiantes e JOIN calificaciones cal ON e.id_estudiante = cal.id_estudiante
GROUP BY e.id_estudiante, e.carnet, e.nombres, e.apellidos
HAVING AVG(cal.nota) >= 90.00;
```

g. Filtrar notas por rango y fecha

```
SELECT * FROM calificaciones
WHERE nota BETWEEN 85.00 AND 100.00 AND fecha_registro >= '2026-09-01';
```

h. Cursos asignados a docentes

```
SELECT CONCAT(d.nombres, ' ', d.apellidos) AS docente, d.especialidad, c.nombre_curso
FROM docentes d JOIN asignacion_docente_curso adc ON d.id_docente = adc.id_docente
JOIN cursos c ON adc.id_curso = c.id_curso;
```

i. Actualizar una calificación específica

```
UPDATE calificaciones SET nota = 96.00 WHERE id_calificacion = 1;
```

j. Eliminar una calificación específica

```
DELETE FROM calificaciones WHERE id_calificacion = 2;
```

Nota: las sentencias UPDATE y DELETE son ejemplos operativos. En la aplicación deben ejecutarse con parámetros, validaciones de permisos y una condición que identifique inequívocamente el registro que se desea modificar o eliminar.

9. CONCLUSIONES

1. El uso de listas enlazadas y estructuras dinámicas permite practicar la gestión de referencias en Java y realizar operaciones de inserción, búsqueda y ordenamiento en memoria, demostrando que el procesamiento algorítmico en memoria reduce la dependencia del motor SQL.

2. La reestructuración de la base de datos resolvió los problemas de inconsistencia y redundancia heredados de proyectos anteriores, alcanzando la Primera Forma Normal (1FN).

3. La integración de patrones DAO y PreparedStatement garantizó una separación adecuada de capas y neutralizó vulnerabilidades de Inyección SQL.

4. El modelo relacional separa usuarios, estudiantes, docentes, niveles académicos, cursos, asignaciones y calificaciones, e incorpora restricciones para mejorar la integridad referencial.

5. La separación entre dominio, estructuras dinámicas y persistencia, mejora la mantenibilidad y la seguridad del sistema.

10. RECOMENDACIONES

1. Evaluar árboles binarios de búsqueda o árboles AVL para optimizar búsquedas cuando el volumen de estudiantes lo justifique.

2. Realizar operaciones de persistencia en segundo plano sin bloquear la interfaz gráfica y gestionar adecuadamente las transacciones.

3. Considerar una API REST para permitir el acceso controlado desde clientes web y móviles.

4. Sustituir todas las contraseñas de prueba por hashes Argon2id o bcrypt generados desde la aplicación; no almacenar contraseñas en texto plano.

11. BIBLIOGRAFÍA

1. Deitel, P., & Deitel, H. (2016). Cómo programar en Java (10.ª ed.). Pearson Educación.

2. Joyanes Aguilar, L., & Zahonero Martínez, I. (2008). Estructuras de datos en Java. McGraw-Hill Interamericana.

3. Gutarra, A. (2023). Análisis, diseño, desarrollo e implementación del sistema de gestión académica integral “SISGACI” para los colegios y academias pre-universitarias del “Consorcio Educativo Makarenko” [Tesis de pregrado, Universidad Nacional Mayor de San Marcos]. Repositorio Institucional - UNMSM. https://cybertesis.unmsm.edu.pe/item/734e424f-d640-43fb-8fc4-10b0cd2f5bbb

4. Ortegón Cortázar, G. (2015). Optimización de sistemas de gestión académica. Una propuesta de gestión, medición y procesamiento de datos en un entorno virtual de aprendizaje para la toma de decisiones en instituciones educativas. Revista EAN, (79), 80–97. https://repository.universidadean.edu.co/entities/publication/39a5b382-34e3-4f1c-9d91-678ca841c180

Universidad Nacional de Ingeniería

12. ANEXOS

ANEXO 1: Modelo de entidad relación